

Multi-Authority Attribute-Based Multi-Keyword Searchable Encryption with Dynamic Membership from Lattices

Er-Shuo Zhuang
Information Security Research
Center
National Sun Yat-sen University
Kaohsiung, Taiwan
zhuanges@gmail.com

Chia-Yu Lin
Department of Computer Science
and Engineering
National Sun Yat-sen University
Kaohsiung, Taiwan
xcvb880224@gmail.com

Chun-I Fan*
Department of Computer Science
and Engineering
National Sun Yat-sen University
Kaohsiung, Taiwan
cifan@mail.cse.nsysu.edu.tw

Arijit Karati
Department of Computer Science and Engineering
National Sun Yat-sen University
Kaohsiung, Taiwan
arijit.karati@mail.cse.nsysu.edu.tw

Debasis Das
Department of Computer Science and Engineering
Indian Institute of Technology Jodhpur
Rajasthan, India
debasis@iitj.ac.in

Abstract—With the growing need for secure cloud storage and the threat of quantum attacks, we propose a post-quantum secure lattice-based multi-authority attribute-based multi-keyword searchable encryption scheme with dynamic membership. Based on the decisional learning with errors assumption, the proposed scheme resists quantum adversaries while supporting flexible multi-keyword search, decentralized authority to avoid key escrow, and dynamic attribute updates without affecting other users. It enables users to independently generate search tokens, allowing offline authorities and reducing communication overhead. Despite higher encryption costs, experimental results demonstrate that search and decryption remain efficient compared to existing post-quantum lattice-based solutions. The proposed scheme achieves strong data confidentiality and keyword privacy in the post-quantum setting.

Index Terms—Lattice-Based Cryptography, Attribute-Based Encryption, Searchable Encryption, Dynamic Membership Management, Multi-Authority, Post-Quantum Cryptography

I. INTRODUCTION

Song et al. [1] proposed the first searchable encryption scheme. Boneh et al. [2] later introduced public-key encryption with keyword search (PEKS), enabling secure search via trapdoors. Meanwhile, Sahai and Waters [3] proposed attribute-based encryption (ABE), which provides fine-grained access control by tying decryption rights to user attributes. ABE has two variants: Key-Policy ABE (KP-ABE) [4] and Ciphertext-Policy ABE (CP-ABE) [5].

Although ABE offers strong access control, it struggles with scalability and dynamic user management. Fan et al. [6] introduced dynamic membership to support attribute revocation. Traditional ABE schemes also depend on a single trusted

key generation center (KGC), leading to centralization and key escrow concerns. Multi-authority ABE (MA-ABE) [7]–[9] mitigates these risks by distributing trust across multiple authorities.

Another pressing issue is the threat posed by quantum computing. Shor's algorithm [10] compromises classical cryptosystems, driving the development of post-quantum cryptography (PQC). Lattice-based cryptography is a leading PQC candidate due to its efficiency and strong security assumptions. Prior work has explored lattice-based ABE [11]–[13], and Liu et al. [14] incorporated keyword search into such frameworks.

A. Contributions

Our proposed scheme addresses these limitations with the following features:

- **Multi-Authority Architecture:** Employs a CP-ABE framework with multiple authorities to eliminate key escrow and enhance trust and scalability.
- **Advanced Functionalities:** Supports multi-keyword search, dynamic membership updates, and offline search token generation, enabling practical and fine-grained access control in cloud environments.
- **Post-Quantum Security:** Leverages hard lattice problems to ensure quantum resistance and collusion resistance.

II. PRELIMINARIES

We introduce some background knowledge here.

A. Lattice Sampling and Trapdoor Algorithms

We introduce some trapdoor generation algorithms and sampling algorithms from Micciancio and Peikert [15].

*Corresponding author: Chun-I Fan.

Definition II.1 (TrapGen). Let $n \in \mathbb{Z}^+$, q be prime, $m = \lceil 6n \log q \rceil$, and $\sigma > 0$. The algorithm **TrapGen**(n, m, q, σ) outputs $(\mathbf{A}, \mathbf{T}_\mathbf{A})$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ is statistically close to uniform and $\mathbf{T}_\mathbf{A} \in \mathbb{Z}^{m \times m}$ is a trapdoor basis of $\Lambda_q^\perp(\mathbf{A})$ with $\|\tilde{\mathbf{T}}_\mathbf{A}\| \leq \sqrt{n \log q}$ with high probability.

Definition II.2 (SamplePre). Given $(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma)$ with $\sigma > \|\tilde{\mathbf{T}}_\mathbf{A}\| \cdot \omega(\sqrt{\log m})$, the algorithm **SamplePre** outputs $\mathbf{e} \in \Lambda_q^\mathbf{u}(\mathbf{A})$ distributed statistically close to $\mathcal{D}_{\Lambda_q^\mathbf{u}(\mathbf{A}), \sigma}$.

Definition II.3 (SampleLeft). Given $(\mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{M} \in \mathbb{Z}_q^{n \times m'}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \sigma)$, the algorithm **SampleLeft** outputs $\mathbf{e} \in \mathbb{Z}^{m+m'}$ sampled from a distribution statistically close to $\mathcal{D}_{\Lambda_q^\mathbf{u}([\mathbf{A}|\mathbf{M}], \sigma)}$ such that $[\mathbf{A}|\mathbf{M}] \cdot \mathbf{e} = \mathbf{u} \bmod q$.

Definition II.4 (SampleR). Given security parameter m , the algorithm **SampleR**(1^m) outputs an invertible matrix $\mathbf{R} \in \mathbb{Z}_q^{m \times m}$ sampled from a distribution $D_{m \times m}$.

Definition II.5 (BasisDel). Given $(\mathbf{A}, \mathbf{R}, \mathbf{T}_\mathbf{A}, \sigma)$, where $\mathbf{R} \in \mathbb{Z}_q^{m \times m}$ is invertible, the algorithm **BasisDel** outputs a trapdoor basis $\mathbf{T}_\mathbf{F}$ for $\Lambda_q^\perp(\mathbf{F})$ where $\mathbf{F} = \mathbf{A}\mathbf{R}^{-1}$.

B. Access Structure

During encryption, the sender specifies an access structure embedded in the ciphertext to control user access. Let S be the universe of attributes and $S_{ID} \subseteq S$ the attribute set of user ID . The access structure τ is represented as a tree whose leaf nodes correspond to attributes in $S_C \subseteq S$, and whose internal nodes are limited to AND or OR gates.

C. Bloom Filter

A Bloom filter is a probabilistic data structure utilized for efficient membership tests of set elements. By utilizing an l -bit array and k hash functions, denoted as $BF(l, k)$, it can rapidly determine whether an element is present in a set or not. A positive result from a Bloom filter query indicates a high probability that the element is present in the set. However, there is a possibility of false positives. The false positive rate is related to the size of the array and the number of hash functions.

D. System Model

The system model of the proposed scheme is shown in Fig. 1. The scheme consists of six roles as follows:

- **Initializer:** One of the attribute authorities. It collects and publishes updated public keys and global parameters. It cannot access master private keys of other authorities.
- **Public Key Bulletin Board (PKB):** Honest-but-curious entity that publishes public keys and parameters.
- **Attribute Authority (AA):** Each AA generates public/master keys and assists in user key generation and updates.
- **Cloud Server (CS):** Honest-but-curious storage and search provider, responsible for storing encrypted data and processing search queries without learning sensitive information.

- **Data Owner (DO):** Encrypts and uploads ciphertexts, defining access policies to ensure only authorized users can access the data.
- **Data User (DU):** Generates search tokens and retrieves ciphertext, then decrypts data if their attributes satisfy the access policy.

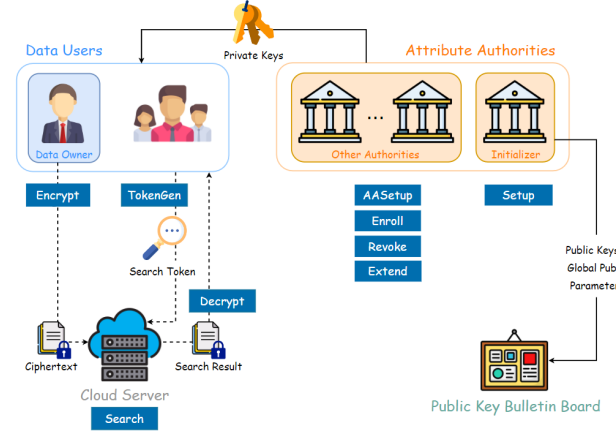


Fig. 1: The System Model

III. CONSTRUCTION

The notations used are summarized in Table I.

TABLE I: The Notations

Notation	Meaning
D	the set of all attribute authorities
U	the set of all attributes
L	the set of all enrolled users' identities
H	a hash function
AA_d	the d -th attribute authority
a_i	the i -th attribute
$ \cdot $	the number of the elements in some set
ID	the identity of some user
χ	a Gaussian distribution
U_d	the set of attributes monitored by AA_d
U^{ID}	the attribute set of ID
U_d^{ID}	the attribute set of ID monitored by AA_d
U_C	the set of attributes associated with CT
D_C	the attribute authorities monitoring U_C
PK_d	the public key of AA_d
MK_d	the master private key of AA_d
SK_d^{ID}	the partial private key of ID for AA_d
SK_{ID}	the private key of ID
W	the keyword set
k_i	the i -th keyword

A. Setup

$GP \leftarrow \text{Setup}(\lambda, D, U)$: Given a security parameter λ , the set of all attribute authorities D , and the set of all attributes U . Then, the Initializer performs the steps below to generate the global public parameters GP :

- 1) Select a prime number q , two positive integers n, m , a positive Gaussian parameter σ , and a Gaussian distribution χ .

- 2) Select a hash function $H_0 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^{n \times m}$.
- 3) Select k hash functions $H_i : \{0, 1\}^* \rightarrow \mathbb{Z}_q$, where $1 \leq i \leq k$.
- 4) Let L denote the set of all enrolled users' identities.
- 5) Randomly choose a vector $\mathbf{u} \in \mathbb{Z}_q^n$.
- 6) Output the global public parameters $GP = \{q, n, m, \sigma, H_0, \{H_i\}_{1 \leq i \leq k}, D, U, L, \chi, \mathbf{u}\}$.

B. AASetup

$(PK_d, MK_d) \leftarrow \text{AASetup}(GP)$: After the Initializer publishes GP , each AA_d performs the following steps to generate their public key PK_d and master secret key MK_d :

- 1) Let U_d denote the set of attributes monitored by AA_d . For each attribute $a_i \in U_d$, run $\text{TrapGen}(n, m, q, \sigma)$ to generate $\mathbf{A}_{d,i} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{T}_{\mathbf{A}_{d,i}} \in \mathbb{Z}_q^{m \times m}$, where a_i denotes the i -th attribute.
- 2) Set $A_d = \{\mathbf{A}_{d,i}\}_{a_i \in U_d}$ and $PK_d = \emptyset$. Publish the public key PK_d .
- 3) Set $\{\mathbf{T}_{\mathbf{A}_{d,i}}\}_{a_i \in U_d}$ as the master secret key MK_d .

Note: Through the AASetup algorithm, each attribute authority independently generates its own key pair, achieving a decentralized structure with the multi-authority feature.

C. Enroll

$(PK_d, SK_d^{ID}) \leftarrow \text{Enroll}(GP, PK_d, MK_d, ID, U_d^{ID})$: The user enrollment is executed by each AA_d involving the user's ID and a set of attributes U_d^{ID} that are monitored by AA_d . The private key for U_d^{ID} is generated by each AA_d through the steps below:

- 1) $\forall a_i \in U_d^{ID}$:
 - a) Invoke $\text{SampleR}(1^m)$ to sample an invertible matrix $\mathbf{R}_{d,i}^{ID} \in \mathbb{Z}_q^{m \times m}$.
 - b) Set a matrix $\mathbf{B}_{d,i}^{ID} = \mathbf{A}_{d,i}(\mathbf{R}_{d,i}^{ID})^{-1}$. Then, run $\text{BasisDel}(\mathbf{A}_{d,i}, \mathbf{R}_{d,i}^{ID}, \mathbf{T}_{\mathbf{A}_{d,i}}, \sigma)$ to generate $\mathbf{T}_{\mathbf{B}_{d,i}^{ID}} \in \mathbb{Z}_q^{m \times m}$.
- 2) $\forall a_i \in U_d - U_d^{ID}$:
 - a) Randomly choose a matrix $\mathbf{B}_{d,i}^{ID} \in \mathbb{Z}_q^{n \times m}$.
- 3) Set $PK_d = PK_d \cup \{\mathbf{B}_{d,i}^{ID}\}_{a_i \in U_d}$.
- 4) Send partial private key $SK_d^{ID} = \{\mathbf{T}_{\mathbf{B}_{d,i}^{ID}}\}_{a_i \in U_d^{ID}}$ to ID .
- 5) If $ID \notin L$, set $L = L \cup ID$.

ID sets its fully private key $SK_{ID} = \{SK_d^{ID}\}_{d \in D}$.

D. Revoke

$(\mathbf{B}_{d,t}^{ID})' \leftarrow \text{Revoke}(GP, PK_d, ID, t)$: The revocation of the attribute is executed by AA_d involving the target attribute t and the user's identity ID . AA_d revokes the target attribute by doing the steps below:

- 1) Randomly choose a matrix $(\mathbf{B}_{d,t}^{ID})' \in \mathbb{Z}_q^{n \times m}$.
- 2) Replace $\mathbf{B}_{d,t}^{ID}$ with $(\mathbf{B}_{d,t}^{ID})'$.

E. Extend

$(PK_d, SK_d') \leftarrow \text{Extend}(GP, PK_d, MK_d, ID, t, SK_d^{ID})$: The extension of the attribute is executed by AA_d involving the target attribute t and the user's identity ID . AA_d generates a new private key of the target attribute for ID by doing the steps below:

- 1) Invoke $\text{SampleR}(1^m)$ to sample an invertible matrix $\mathbf{R}_{d,t}^{ID} \in \mathbb{Z}_q^{m \times m}$.
- 2) Set a matrix $(\mathbf{B}_{d,t}^{ID})' = \mathbf{A}_{d,t}(\mathbf{R}_{d,t}^{ID})^{-1}$. Then, run $\text{BasisDel}(\mathbf{A}_{d,t}, \mathbf{R}_{d,t}^{ID}, \mathbf{T}_{\mathbf{A}_{d,t}}, \sigma)$ to generate $\mathbf{T}_{(\mathbf{B}_{d,t}^{ID})'} \in \mathbb{Z}_q^{m \times m}$.
- 3) Replace $\mathbf{B}_{d,t}^{ID}$ with $(\mathbf{B}_{d,t}^{ID})'$.
- 4) Send partial private key $(SK_d^{ID})' = SK_d^{ID} \cup \{\mathbf{T}_{(\mathbf{B}_{d,t}^{ID})'}\}$ to ID .

ID updates its private key $SK_{ID} = SK_{ID} \cup (SK_d^{ID})'$.

Note: The Enroll, Revoke, and Extend algorithms collectively enable dynamic membership management.

F. Encrypt

$CT \leftarrow \text{Encrypt}(GP, b, k_0, W, \tau, \{PK_d\}_{d \in D_C})$: DO encrypts the message $b \in \{0, 1\}$ with a selected access tree τ through the steps below:

- 1) Randomly select a noise value $x \in \chi$.
- 2) Select a combination of operation gates and a set of attributes $U_C = \{a_i\}_{a_i \in \tau}$ according to the access tree τ .
- 3) For each attribute $a_i \in U_C$, randomly choose a noise vector $\mathbf{x}_{1,d,i} \in \chi^m$.
- 4) Randomly select a value $r \in \mathbb{Z}_q$ and a vector $\mathbf{s}_1 \in \mathbb{Z}_q^n$ and compute
$$C = \mathbf{u}^T \mathbf{s}_1 \cdot r + b \lfloor \frac{q}{2} \rfloor + x \in \mathbb{Z}_q. \quad (1)$$
- 5) The value of the root node in the access tree τ is assigned as r . Then, beginning from the root node, each leaf node is assigned a value r_i according to the operation gate in τ following the method defined in Section II-B.
- 6) For each attribute $a_i \in U_C$ and each $ID \in L$, compute the partial ciphertext as follows:

$$C_{d,i}^{ID} = (\mathbf{B}_{d,i}^{ID})^T \mathbf{s}_1 \cdot r_i + \mathbf{x}_{1,d,i} \in \mathbb{Z}_q^m. \quad (2)$$

- 7) Select a category of the keyword set $k_0 \in \{0, 1\}$, and compute $\mathbf{K} = H_0(k_0) \in \mathbb{Z}_q^{n \times m}$.
- 8) Select a keyword set $W = \{k_1, k_2, \dots, k_j\}$, where each $k_i \in \{0, 1\}^*$ and the last bit of each $k_i = k_0$.
- 9) Generate a binary vector $\mathbf{w} = (w_1, w_2, \dots, w_l)$ for W using k hash functions of the bloom filter $BF(l, k)$ defined in Section II-C, where each $w_i \in \{0, 1\}$.
- 10) Randomly select a noise $x_l \in \chi$, noise vectors $\mathbf{x}_{2,d,i} \in \chi^m$ and a vector $\mathbf{s}_2 \in \mathbb{Z}_q^n$. For $\theta = 1$ to l , compute

$$I_{1,\theta} = \mathbf{u}^T \mathbf{s}_2 \cdot r + w_\theta \lfloor \frac{q}{2} \rfloor + x_l \in \mathbb{Z}_q. \quad (3)$$

TABLE II: The Properties Comparison

	Multi-Authority	Multi-Keyword	Authentication		Collusion Resistance	KGC/Authorities Offline
			Search	Decrypt		
Yang <i>et al.</i> [16]	✓			✓	✗	
Sun <i>et al.</i> [17]	✓			✓	✗	
Zhuang <i>et al.</i> [18]	✓			✓	✓	
Varri <i>et al.</i> [19]	✗	✓	✓	✗	✗	✗
Wang <i>et al.</i> [20]	✗	✗	✓	✗	✗	✓
Li <i>et al.</i> [21]	✗	✗	✗	✓	✗	✗
Chen [22]	✗	✗	✓	✓	✓	✓
Wang [23]	✗	✓	✓	✓	✓	✓
Ours	✓	✓	✓	✓	✓	✓

✓ indicates that the scheme provides the property.

✗ indicates that the scheme does not provide the property.

\ indicates that the application scenario of the scheme does not consider searchable encryption, so the property is not needed.

11) For each attribute $a_i \in U_C$ and each $ID \in L$, compute

$$I_{d,i}^{ID} = [\mathbf{B}_{d,i}^{ID} | \mathbf{K}]^T \mathbf{s}_2 \cdot r_i + \mathbf{x}_{2,d,i} \in \mathbb{Z}_q^{2m}. \quad (4)$$

12) Set the index $I^{ID} = \{\{I_{1,\theta}\}_{1 \leq \theta \leq l}, \{I_{d,i}^{ID}\}_{a_i \in U_C}\}$.

13) Set the ciphertext $CT = \{C, \{C_{d,i}^{ID}\}_{a_i \in U_C, ID \in L}\}$.

14) Upload $(CT, \{I^{ID}\}_{ID \in L}, \tau)$ to CS.

Note: Each ID has independent keys, ensuring collusion resistance. Both messages and keywords are encrypted, so user attributes are verified during decryption and search.

G. TokenGen

$(\mathbf{w}', \{\mathbf{k}_{d,i}^{ID}\}_{a_i \in U^{ID}}) \leftarrow \text{TokenGen}(GP, k'_0, W', \{PK_d\}_{d \in D_C}, SK_{ID})$: DU generate the search token as follows:

- 1) Select a category of the keyword set $k'_0 \in \{0, 1\}$, and compute $\mathbf{K}' = H_0(k'_0) \in \mathbb{Z}_q^{n \times m}$.
- 2) Select a keyword set $W' = \{k'_1, k'_2, \dots, k'_j\}$, where each $k'_i \in \{0, 1\}^*$ and the last bit of each $k'_i = k'_0$.
- 3) Compute a binary vector $\mathbf{w}'$ for W' using k hash functions of the bloom filter $BF(l, k)$.
- 4) For each attribute $a_i \in U^{ID}$, run $\text{SampleLeft}(\mathbf{B}_{d,i}^{ID}, \mathbf{K}', \mathbf{T}_{\mathbf{B}_{d,i}^{ID}}, \mathbf{u}, \sigma)$ to generate $\mathbf{k}_{d,i}^{ID} \in \mathbb{Z}_q^{2m}$, where $[\mathbf{B}_{d,i}^{ID} | \mathbf{K}'] \mathbf{k}_{d,i}^{ID} = \mathbf{u}$ and U^{ID} presents as the attribute set of user ID .
- 5) Send $(\mathbf{w}', \{\mathbf{k}_{d,i}^{ID}\}_{a_i \in U^{ID}})$ to CS.

Note: Since the DU can independently generate tokens, the scheme achieves an offline KGC architecture.

H. Search

$CT/\perp \leftarrow \text{Search}(\mathbf{w}', \{\mathbf{k}_{d,i}^{ID}\}_{a_i \in U^{ID}}, \{I^{ID}\}_{ID \in L}, \tau)$: CS receives the search token $(\mathbf{w}', \{\mathbf{k}_{d,i}^{ID}\}_{a_i \in U^{ID}})$ from DU, and do the steps below:

- 1) For $\theta = 1$ to l , compute w'_θ based on the operation gate in τ , for example:
 - When τ is constructed by only **AND** gates, compute $w'_\theta = I_{1,\theta} - \sum_{a_i \in U_C} (\mathbf{k}_{d,i}^{ID})^T I_{d,i}^{ID}$.

- When τ is constructed by only **OR** gates, compute $w'_\theta = I_{1,\theta} - (\mathbf{k}_{d,i}^{ID})^T I_{d,i}^{ID}$, where $a_i \in U_C$.

- 2) If $|w'_\theta - \lfloor \frac{q}{2} \rfloor| < \lfloor \frac{q}{4} \rfloor$, output $w'_\theta = 1$; otherwise, output $w'_\theta = 0$.
- 3) Compute the bitwise-or. If $\mathbf{w}' \cup \bar{\mathbf{w}} = \bar{\mathbf{w}}$, then the keyword set $W' \subset W$.
- 4) If $\mathbf{w}' \cup \bar{\mathbf{w}} = \bar{\mathbf{w}}$, send the corresponding ciphertext CT to DU. Otherwise, return $\perp$.

I. Decrypt

$b' \leftarrow \text{Decrypt}(CT, GP, \{PK_d\}_{d \in D_C}, SK_{ID})$: Upon receiving the ciphertext CT , any user whose attribute set meets the requirements of τ can decrypt the ciphertext by doing the steps below:

- 1) For each attribute $a_i \in U_C$, run $\text{SamplePre}(\mathbf{B}_{d,i}^{ID}, \mathbf{T}_{\mathbf{B}_{d,i}^{ID}}, \mathbf{u}, \sigma)$ to generate $\mathbf{e}_{d,i}^{ID} \in \mathbb{Z}_q^m$, where $\mathbf{B}_{d,i}^{ID} \mathbf{e}_{d,i}^{ID} = \mathbf{u}$.
- 2) Compute b' based on the operation gate in τ , for example:
 - When τ is constructed by only **AND** gates, compute $b' = C - \sum_{a_i \in U_C} (\mathbf{e}_{d,i}^{ID})^T C_{d,i}^{ID}$.
 - When τ is constructed by only **OR** gates, compute $b' = C - (\mathbf{e}_{d,i}^{ID})^T C_{d,i}^{ID}$, where $a_i \in U_C$.
- 3) If $|b' - \lfloor \frac{q}{2} \rfloor| < \lfloor \frac{q}{4} \rfloor$, output $b' = 1$; otherwise, output $b' = 0$.

Note: To adapt the proposed scheme for transmitting multiple bits, the encryption process can generate multiple distinct ciphertexts C , with each C carrying one bit, while other components, such as $(\mathbf{e}_{d,i}^{ID})^T$ and $C_{d,i}^{ID}$, can be reused.

IV. COMPARISONS

We compare the proposed scheme with the post-quantum searchable encryption schemes in [19]–[23]. Unlike these schemes, our design supports a multi-authority setting with independent key pairs, dynamic membership via **Enroll**, **Revoke**, and **Extend**, offline token generation to reduce

TABLE III: Comparison Parameters

	Description	Value
n	Security parameter	284
m	Basis dimension	13, 812
q	Modulo	2^{24}
l	Size of attribute set U_C	10
l'	Number of bits of a set used in subset predicate encryption (SPE)	10
u	Total number of users	50
w	Total number of keywords in CT	5
h	Total number of hash functions	3
k	Length of bloom filter array	32

TABLE IV: Ciphertext, Index, and Token Length

	Ciphertext	Index	Token
Varri <i>et al.</i> [19]	$(l + n) \mathbb{Z}_q $	$(l + 2mw) \mathbb{Z}_q $	$(2mw + 2lmn) \mathbb{Z}_q $
Wang <i>et al.</i> [20]	$(l'm + m + 1) \mathbb{Z}_q $	$(l'm + m) \mathbb{Z}_q $	$2hm \mathbb{Z}_q $
Li <i>et al.</i> [21]	$2 \mathcal{R}_q + n \mathbb{Z}_q $	$(w + 1) \mathcal{R}_q + n \mathbb{Z}_q $	$ \mathcal{R}_q $
Chen [22]	$(lmu + 1) \mathbb{Z}_q $	$(lmu + 1) \mathbb{Z}_q $	$hlm \mathbb{Z}_q $
Wang [23]	$(lmu + 1) \mathbb{Z}_q $	$(lmu + h + w) \mathbb{Z}_q $	$hlm \mathbb{Z}_q $
Ours	$(lmu + 1) \mathbb{Z}_q $	$(2lmu + k) \mathbb{Z}_q $	$2lm \mathbb{Z}_q $

TABLE V: The Notations and Operations Cost

Notation	The computation time of	Cost
T_{mul1}	$(\mathbb{Z}_q \times \mathbb{Z}_q)$	0.0012(ms)
T_{mul2}	$(\mathbb{Z}_q \times \mathbb{Z}_q^m)$	0.0015(ms)
T_{mul3}	$(\mathbb{Z}_q^n \times \mathbb{Z}_q^n)$	0.002(ms)
T_{mul4}	$(\mathbb{Z}_q^{1 \times n} \times \mathbb{Z}_q^{n \times 1})$	0.003(ms)
T_{mul5}	$(\mathbb{Z}_q^{1 \times m} \times \mathbb{Z}_q^{m \times 1})$	0.012(ms)
T_{mul6}	$(\mathbb{Z}_q^{n \times 2m} \times \mathbb{Z}_q^{2m \times 1})$	6.284(ms)
T_{mul7}	$(\mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^{n \times 1})$	3.077(ms)
T_{mul8}	$(\mathbb{Z}_q^{1 \times 2m} \times \mathbb{Z}_q^{2m \times 1})$	0.018(ms)
T_{mul9}	$(\mathbb{Z}_q \times \mathbb{Z}_q^{2m})$	0.273(ms)
T_{mul10}	$(\mathbb{Z}_q^{2m \times n} \times \mathbb{Z}_q^{n \times 1})$	6.101(ms)
T_{mulR}	$(\mathcal{R}_q \times \mathcal{R}_q)$	0.952(ms)
T_H	SHA-512 for inputs 128 bits	0.016(ms)

TABLE VI: Computation Cost of Encryption

	Encryption	Searching	Decryption
[19]	$w * (T_{mul4} + T_H)$ $\approx 0.095(\text{ms})$	$T_{mul1} + T_{mul3} + 2 * T_{mul6}$ $\approx 12.5712(\text{ms})$	$T_{mul3} + T_{mul6}$ $\approx 6.286(\text{ms})$
[20]	$(l' * w + 1) * T_{mul4} + (l' + 1) * T_{mul7} + (l' * w) * T_H$ $\approx 34.8(\text{ms})$	$l' * T_{mul8}$ $\approx 0.18(\text{ms})$	T_{mul8} $\approx 0.018(\text{ms})$
[21]	$(3 + w) * T_{mulR} + l * T_H$ $\approx 9.568(\text{ms})$	$T_{mulR} + T_H$ $\approx 1.096(\text{ms})$	$T_{mulR} + l * T_H$ $\approx 1.24(\text{ms})$
[22]	$(2 * u) * (T_{mul1} + T_{mul4}) + (2 * l * u) * T_{mul7} + h * T_H$ $\approx 3077.468(\text{ms})$	$(h * l) * T_{mul5}$ $\approx 0.36(\text{ms})$	$(2 * l) * T_{mul5}$ $\approx 0.24(\text{ms})$
[23]	$(2 * l * u) * (T_{mul1} + T_{mul4} + T_{mul7}) + h * (T_{mul1} + T_{mul4} + T_H) + w * T_H$ $\approx 3081.3406(\text{ms})$	$(2 * h) * (l * T_{mul2} + T_{mul4})$ $\approx 0.108(\text{ms})$	$(2 * l) * T_{mul5}$ $\approx 0.24(\text{ms})$
Ours	$(w * h + 1) * T_H + 2 * (T_{mul1} + T_{mul4}) + (l * u) * (T_{mul2} + T_{mul7}) + (l * u) * (T_{mul9} + T_{mul10})$ $\approx 4741.296(\text{ms})$	$l * T_{mul8}$ $\approx 0.18(\text{ms})$	$l * T_{mul5}$ $\approx 0.12(\text{ms})$

overhead, and collusion resistance through identity-specific keys and dual encryption on messages and keywords. Table II summarizes their properties. The parameter settings for comparison are provided in Table III.

A. Transmission Cost

Table IV compares ciphertext, index, and token lengths. Let U_C be the set of attributes in the ciphertext ($|U_C| = l$), u the total number of users, w the number of keywords, h the

number of hash functions, and k the bloom filter array length. Chen's [22], Wang's [23], and our scheme multiply ciphertext size by u , increasing storage but preventing user collusion, unlike other schemes.

B. Computation Cost

Table V and Table VI show computation costs for encryption, search, and decryption, measured on a Windows 11 Home system (Intel i7-9750H, 8GB RAM, Python 3.9.5, Numpy 1.20.3). Encryption in Chen's [22], Wang's [23], and our scheme scales with u , increasing time (up to 4741.29 ms) but strengthening security and enabling dynamic membership. Search and decryption remain efficient, with our scheme achieving the shortest decryption time among schemes supporting multi-keyword search. Additionally, while some schemes require KGC assistance for token generation, ours and Wang et al. [20], Chen [22], and Wang [23] support offline KGC, enhancing flexibility. Overall, our scheme balances richer functionality with practical performance.

V. CONCLUSIONS

In this study, we propose a lattice-based ABE scheme supporting multi-authority architecture, multi-keyword search, and dynamic membership management, achieving post-quantum security under the D-LWE assumption. By decentralizing authority across multiple key generators, the scheme mitigates key escrow, enhances trust, and resists collusion. Multi-keyword search improves query flexibility, while attribute independence and revocation enable fine-grained, dynamic access control. Although encryption incurs higher overhead, search and decryption remain efficient and practical. Formal security guarantees under IND-CPA and IND-CKA models are provided, with full proofs available upon request.

ACKNOWLEDGMENT

This work was partially supported by the National Science and Technology Council (NSTC) of Taiwan under grant 114-2221-E-110-040. It also was supported by the MediaTek Advanced Research Center under the research contract MTKC-2024-1133. Additional financial support was provided by the Information Security Research Center at National Sun Yat-sen University in Taiwan.

REFERENCES

- [1] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proceeding 2000 IEEE symposium on security and privacy. S&P 2000*. IEEE, 2000, pp. 44–55.
- [2] D. Boneh, G. Di Crescenzo, R. Ostrovsky, and G. Persiano, "Public key encryption with keyword search," in *Advances in Cryptology-EUROCRYPT 2004: International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004. Proceedings 23*. Springer, 2004, pp. 506–522.
- [3] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *Advances in Cryptology-EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings 24*. Springer, 2005, pp. 457–473.
- [4] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *Proceedings of the 13th ACM conference on Computer and communications security*, 2006, pp. 89–98.
- [5] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *2007 IEEE symposium on security and privacy (SP'07)*. IEEE, 2007, pp. 321–334.
- [6] C.-I. Fan, V. S.-M. Huang, and H.-M. Ruan, "Arbitrary-state attribute-based encryption with dynamic membership," *IEEE Transactions on Computers*, vol. 63, no. 8, pp. 1951–1961, 2013.
- [7] M. Chase, "Multi-authority attribute based encryption," in *Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007. Proceedings 4*. Springer, 2007, pp. 515–534.
- [8] H. Lin, Z. Cao, X. Liang, and J. Shao, "Secure Threshold Multi Authority Attribute Based Encryption without a Central Authority," *Progress in Cryptology-INDOCRYPT 2008*, p. 426, 2008.
- [9] A. Lewko and B. Waters, "Decentralizing attribute-based encryption," in *Advances in Cryptology-EUROCRYPT 2011: 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15-19, 2011. Proceedings 30*. Springer, 2011, pp. 568–588.
- [10] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 1994, pp. 124–134.
- [11] S. Agrawal, X. Boyen, V. Vaikuntanathan, P. Voulgaris, and H. Wee, "Fuzzy identity based encryption from lattices," *Cryptology ePrint Archive*, 2011.
- [12] J. Zhang and Z. Zhang, "A ciphertext policy attribute-based encryption scheme without pairings," in *Proceedings of the 7th international conference on Information Security and Cryptology*, 2011, pp. 324–340.
- [13] X. Boyen, "Attribute-based functional encryption on lattices," in *Theory of Cryptography: 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3-6, 2013. Proceedings*. Springer, 2013, pp. 122–142.
- [14] L. Liu, S. Wang, B. He, and D. Zhang, "A keyword-searchable ABE scheme from lattice in cloud storage environment," *Ieee Access*, vol. 7, pp. 109 038–109 053, 2019.
- [15] D. Micciancio and C. Peikert, "Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller," in *Eurocrypt*, vol. 7237. Springer, 2012, pp. 700–718.
- [16] Y. Yang, J. Sun, Z. Liu, and Y. Qiao, "Practical revocable and multi-authority CP-ABE scheme from RLWE for Cloud Computing," *Journal of Information Security and Applications*, vol. 65, p. 103108, 2022.
- [17] J. Sun, Y. Qiao, Z. Liu, Y. Chen, and Y. Yang, "Practical Multi-Authority Ciphertext Policy Attribute-Based Encryption from R-LWE," in *2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*. IEEE, 2021, pp. 1435–1443.
- [18] E.-S. Zhuang, C.-I. Fan, and I.-H. Kuo, "Multiauthority Attribute-Based Encryption With Dynamic Membership From Lattices," *IEEE Access*, vol. 10, pp. 58 254–58 267, 2022.
- [19] U. S. Varri, S. K. Pasupuleti, and K. Kadambari, "CP-ABSEL: Ciphertext-policy attribute-based searchable encryption from lattice in cloud storage," *Peer-to-Peer Networking and Applications*, vol. 14, pp. 1290–1302, 2021.
- [20] P. Wang, B. Chen, T. Xiang, and Z. Wang, "Lattice-based public key searchable encryption with fine-grained access control for edge computing," *Future Generation Computer Systems*, vol. 127, pp. 373–383, 2022.
- [21] C. Li, M. Dong, J. Li, G. Xu, X.-B. Chen, W. Liu, and K. Ota, "Efficient medical big data management with keyword-searchable encryption in healthchain," *IEEE Systems Journal*, vol. 16, no. 4, pp. 5521–5532, 2022.
- [22] X.-J. Chen, "Lattice-Based Searchable Attribute-Based Encryption Supporting Dynamic Membership Management," Master thesis, National Sun Yat-sen University, 2021.
- [23] D.-R. Wang, "Multi-Keyword Searchable Attribute-Based Encryption Supporting Dynamic Membership Management from Lattices," Master thesis, National Sun Yat-sen University, 2022.